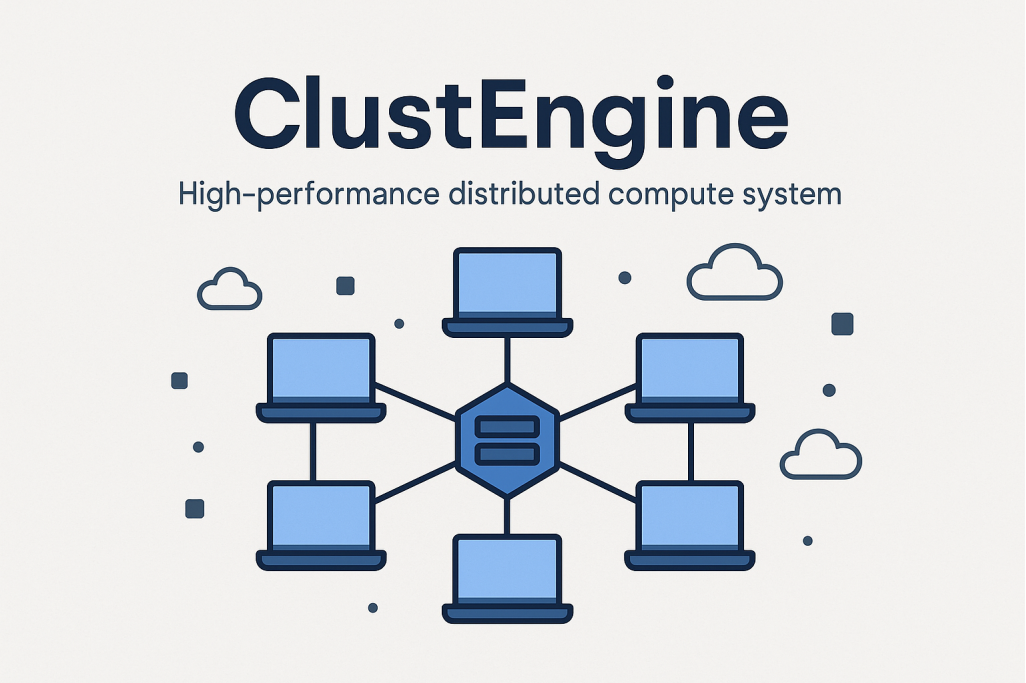
Technical Documentation



Professors: Diego Passos, Fernanda Passos
Student: Bernardo Miguel Lopo Silva

Table of contents

1 Introduction

1.1 Project Objective

The primary objective of the **ClustEngine** project is to provide a comprehensive, automated, and seamless solution for deploying, configuring, and managing cloud based computational clusters. This system is designed to offer a comprehensive control over the cluster infrastructure while abstracting away the low-level complexity typically associated with distributed system setups.

Specifically, the project aims to:

- **Automate the provisioning of instances** on a vast list of cloud Web Services.
- **Establish secure and scalable connectivity** between nodes to ensure effective communication and file sharing.
- **Set up a consistent software environment** across all nodes, including the installation of necessary tools such, for example, OpenMPI.
- **Enable the seamless deployment and execution of distributed applications**, across the cluster using parallel computing.
- **Provide centralized control mechanisms** to start, stop, delete, and monitor the entire cluster infrastructure in a reproducible and maintainable manner.

This project seeks to simplify the complex process of managing multiple nodes in cloud environments, making it accessible to developers, researchers, and engineers who require robust and automated distributed computing capabilities without being experts in cloud infrastructure.

1.2 Use Cases

ClustEngine is designed to serve a variety of use cases where scalable, cloud-based computing clusters are required. Common scenarios include:

- **Scientific Computing:** Running high-performance simulations or computations (e.g., ThermionsApp) that require distributed execution.
- **Educational Environments:** Demonstrating parallel computing concepts and cloud infrastructure orchestration in academic or training settings.
- **Benchmarking and Prototyping:** Quickly spinning up temporary clusters to test the performance of algorithms, frameworks, or configurations.
- **CI/CD in HPC Contexts:** Integrating distributed tests and performance validation into continuous integration pipelines.
- **Custom Research Projects:** Supporting computational experiments in fields such as physics, bioinformatics, data analysis, or machine learning.

These use cases are supported by ClustEngine's flexibility and portability philosophy.

1.3 Project Status

As of this version, the ClustEngine project is in an active development phase with a functional and tested prototype available. Key components already implemented include:

- EC2 cluster provisioning via Kotlin and AWS SDK.
- Secure inter-node communication through SSH.
- File sharing via NFS and multi-node configuration scripts.
- Full support for OpenMPI-based application execution.
- Basic CLI-based control and execution workflow.

Ongoing efforts are focused on:

- Improving modularity and configuration options.
- Enhancing logging, error handling, and diagnostics.
- Writing comprehensive documentation and usage guides.
- Adding support for monitoring and resource cleanup.

A pre release version is available on GitHub for testing.

2 Requirements

2.1 Local Environment

ClustEngine was design to be the most portable possible, to ensure that requirement it is required the use of java virtual machine.

2.1.1 Java / AWS CLI

The local environment must include the following tools, properly installed and available in the system `PATH`:

- **Java SDK 21 or higher** – Required to build and run the Kotlin application.
- **AWS CLI v2+** – Required for credential configuration and command-line testing of AWS functionality.

Use the following commands to verify the installation:

```
java --version
aws --version
```

All commands should return a valid version string without errors.

2.2 Cloud Environment

Supported Cloud Providers At present, ClustEngine supports deployment and management exclusively on Amazon Web Services.

2.2.1 AWS Account

An active AWS account is required. The user must have access to the EC2 service and sufficient permissions to:

- Launch, stop, and terminate EC2 instances.
- Create and attach security groups.
- Manage SSH key pairs and Elastic IPs.
- Configure IAM roles and policies.

2.2.2 IAM Permissions and Policies

The account or user executing ClustEngine must be associated with an IAM role or user that has the following permissions at a minimum:

- `ec2:RunInstances`
- `ec2:TerminateInstances`

- `ec2:DescribeInstances`
- `ec2:CreateSecurityGroup`
- `ec2:AuthorizeSecurityGroupIngress`
- `ec2:CreateKeyPair`, `ec2:ImportKeyPair`

2.2.3 SSH Key Pairs

ClustEngine depends on key based SSH authentication to configure and access EC2 nodes. Limitations include:

- Only one key pair can be associated per EC2 instance during launch.
- Key pair names must be unique per AWS region.
- The private key must be stored securely and remain accessible from the machine executing the orchestration scripts.
- Loss of the private key prevents further access to running instances.

It is recommended to either:

- Generate a new key pair for each cluster deployment, or
- Reuse a predefined secured key with limited access.

3 Installation

This section provides a step by step guide to installing and running the ClustEngine project. ClustEngine is a Java based tool that automates the deployment and configuration of a cloud-based cluster using AWS EC2 instances, with support for NFS, OpenMPI, and other distributed computing tools.

3.1 Clone the Repository

For this step, ensure that you have Git installed and access to the internet to clone the repository.

To get started, clone the ClustEngine repository:

```
git clone https://github.com/bernardo-lopo/ClustEngine.git
cd ClustEngine
```

3.2 Initial Configuration

Before building or running the application, you must configure a few project parameters to match your cloud environment and application requirements.

3.2.1 Configuration Files (`config/`)

All configurable settings are located inside the `config/` directory. This folder contains plain-text configuration files that determine:

- AWS region and availability zones
- EC2 instance types and counts
- Security group and key pair settings
- Application-specific parameters (e.g., MPI command line)
- NFS setup and mount points

Modify these files according to your desired infrastructure and workload specifications.

3.2.2 Supported Parameters

The following parameters are typically supported in the configuration files:

- `AWS_REGION`: The region in which to deploy EC2 instances (e.g., `eu-west-1`)
- `INSTANCE_TYPE`: Type of EC2 instances (e.g., `t3.medium`)
- `KEY_PAIR_NAME`: AWS EC2 key pair name to be used for SSH access
- `SECURITY_GROUP_ID`: Predefined security group ID allowing required ports

- `APP_ENTRY_COMMAND`: MPI command or script to execute distributed applications
- `NFS_MOUNT_DIR`: Directory to mount the shared NFS volume on worker nodes

Refer to the inline comments inside each configuration file for more details.

3.2.3 Environment Variables

Some AWS-related credentials and environment-specific values must be available either in your shell or through the AWS CLI:

- `AWS_ACCESS_KEY_ID`
- `AWS_SECRET_ACCESS_KEY`
- `AWS_SESSION_TOKEN` (if using temporary credentials)

These can be exported manually or automatically managed using tools such as the AWS CLI or SDK credential providers.

```
export AWS_ACCESS_KEY_ID=...
export AWS_SECRET_ACCESS_KEY=...
```

3.3 Build with Gradle

To compile the application, use the Gradle wrapper script provided in the root of the repository:

```
./gradlew build
```

This will download dependencies and generate a JAR file in the `build/libs` directory.

3.4 Run the Application

After building the project, you can run the application using the generated JAR file.

3.4.1 `ConsoleAPP.jar`

Execute the application with:

```
java -jar ConsoleAPP.jar
```

Make sure that Java 21 or higher is installed and accessible through your system's `PATH`.

3.4.2 Command Line Parameters

As of this version, the application does not require command line arguments. All configuration is handled through the files in `config/` and environment variables. Future versions may introduce CLI flags for specific behaviors or advanced debugging.

4 Cluster Configuration

This section details how to configure the cloud cluster before deployment. The configuration relies on a properties file located under `config/`) which defines the topology, networking, storage, and software setup for the instances.

4.0.1 Number of Nodes

The number of instances to be launched is defined via:

```
cluster.size = Nº
Ex: cluster.size = 4
```

This value determines how many worker nodes will be deployed in your cluster. The minimum viable number is 2 (1 master, 1 worker).

4.0.2 Instance Type

Specify the instance type for all cluster nodes using:

```
default.instanceType = Type
Ex: default.instanceType = t2.micro
```

Ensure the instance type supports your workload (e.g., CPU/memory requirements for OpenMPI). Common instance types include `t2.micro`, `t3.medium`, `m5.large`, etc.

4.0.3 AWS Region

Define the AWS region where the cluster will be deployed:

```
default.region = eu-central-1
```

This should match the region in which your VPC, subnet, and key pairs are configured. Supported regions include `us-east-1`, `eu-west-1`, `eu-central-1`, etc.

5 How to use the application

5.1 How to execute the app

To execute the app, open a terminal and execute the command:

```
java -jar ConsoleAPP.jar
```

Ensure that the command line points to the ConsoleAPP.jar file location.

5.2 Use cases

As of today in the app main menu, there are a total of 8 use cases:

- Exit
- Init Cluster
- Start Cluster
- Stop Cluster
- Delete Cluster
- Start Instance
- Stop Instance
- Delete Instance

Each case has corresponding number, starting from 0 to 7.

To exit the application gracefully, type 0. To initiate the cluster, type 1. If the cluster is already initiated, an error message will be displayed.

5.2.1 Cluster Actions

If the cluster has been initiated and is stopped, type 2 to start it. If the cluster is running and you want to stop it, type 3. If the cluster is either stopped or running and you want to delete it, type 4.

5.3 Instance Actions

For each instance actions, after selecting the action is necessary to provide the instance ID.

To start an instance, type 5. To stop an instance, type 6. To delete an instance, type 7.

6 Troubleshooting

6.1 Common Issues

6.1.1 SSH Failures

6.1.2 NFS Mount Errors

6.1.3 `mpirun` Failures

6.2 Useful Logs

6.3 Diagnostic Tools

6.4 FAQ

7 Technical Reference

7.1 Project Structure

7.2 APIs and SDKs Used

7.3 Configuration File Details

7.4 Libraries and Versions

7.5 UML Diagram

8 Contribution

8.1 How to Contribute

8.2 Code Standards

8.3 Issues, Pull Requests

8.4 Contributor Requirements

8.5 Contact the Maintainer

9 Appendices and Examples

9.1 Configuration Examples (`config/`)

9.2 Screenshots and Topology Diagrams

9.3 External References (AWS, OpenMPI Docs)

This project development relies on multiple external documentation. The following references were essential to get to a functional and scalable system:

- **Amazon Web Services (AWS):** The official documentation was used extensively for managing EC2 instances, configuring security groups, key pairs, and working with the AWS Systems Manager (SSM). Key references include:
  - Getting Started with Amazon EC2
  - AWS Systems Manager Documentation
  - AWS CLI Configuration
- **OpenMPI:** The OpenMPI documentation was used to configure and execute parallel jobs across nodes, including setting up host files, slot management, and using 'mpirun' with specific options. Key references include:
  - OpenMPI Official Documentation
  - OpenMPI FAQ
  - mpirun(1) Manual Page

These resources provided foundational guidance and troubleshooting support throughout the implementation and testing phases.